VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

WEEK-1

Q. Simulate the following CPU scheduling algorithms a) FCFS b) SJF

1) a) FCFS(First Come First Serve) Scheduling algorithm without arrival times.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the burst time (BT) for each process.
4. Assign process IDs (P1, P2, ..., Pn).
5. Set the waiting time of the first process:
6. WT[0] = 0
7. Calculate the waiting time for the remaining processes:
8. WT[i] = WT[i-1] + BT[i-1]
9. Calculate the turnaround time for each process:
10. TAT[i] = WT[i] + BT[i]
11. Calculate the average waiting time:
12. Average WT = (Sum of all WT) / n
13. Calculate the average turnaround time:
14. Average TAT = (Sum of all TAT) / n
15. Display the Process ID, Burst Time, Waiting Time, and Turnaround Time.
16. Display the Average Waiting Time and Average Turnaround Time.
17. Stop.

Program:

```
#include <stdio.h>

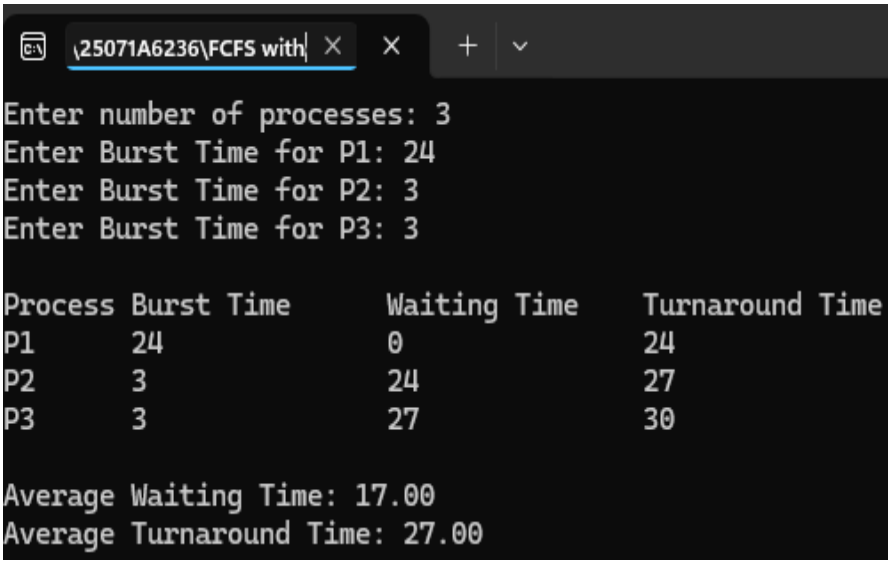
int main()
{
    int n, i;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    int bt[n], wt[n], tat[n];
    float avg_wt = 0, avg_tat = 0;
    for(i = 0; i < n; i++)
    {
        printf("Enter Burst Time for Process P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    wt[0] = 0;
    for(i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    for(i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
```

```
    }

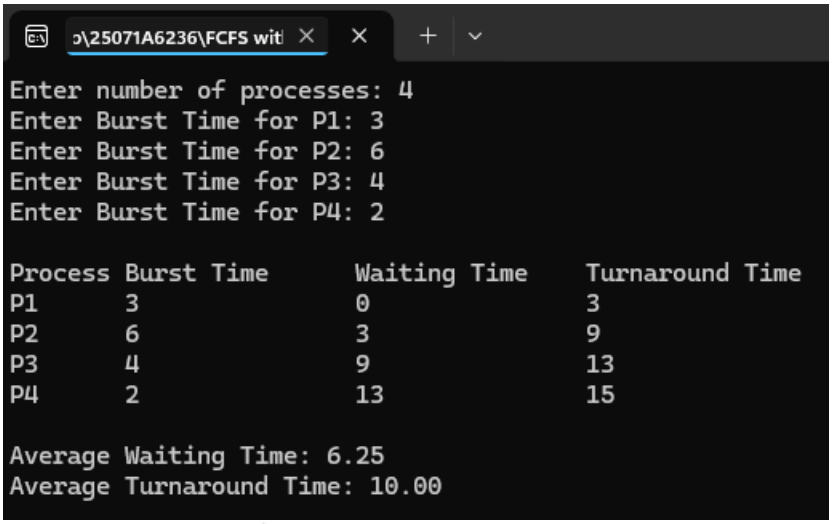
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t\t%d\t\t%d\n",
                i + 1, bt[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);
    return 0;
}
```

OUTPUT:

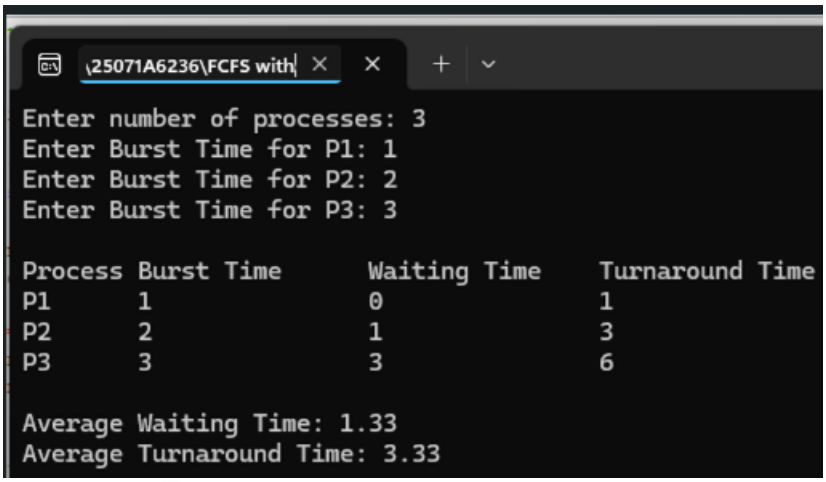
TESTCASE 1:



TESTCASE 2:



TESTCASE 3:



1) b) FCFS(First Come First Serve) Scheduling algorithm with arrival times.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT) and Burst Time (BT) for each process.
4. Assign process IDs (P1, P2, ..., Pn).
5. Sort the processes in ascending order of Arrival Time.
6. If two processes have the same arrival time, execute them in the order they were entered.
7. Set the completion time of the first process:
8. If AT[0] = 0, then CT[0] = BT[0].
9. Otherwise, CT[0] = AT[0] + BT[0].
10. For each remaining process (from i = 1 to n - 1):
11. If CT[i-1] >= AT[i]
    a. CT[i] = CT[i-1] + BT[i]
12. Else (CPU is idle)
    a. CT[i] = AT[i] + BT[i]
13. Calculate the Turnaround Time (TAT) for each process:
14. TAT[i] = CT[i] - AT[i]
15. Calculate the Waiting Time (WT) for each process:
16. WT[i] = TAT[i] - BT[i]
17. Calculate the Average Waiting Time:
18. Average WT = (Sum of all WT) / n
19. Calculate the Average Turnaround Time:
20. Average TAT = (Sum of all TAT) / n
21. Display the Process ID, Arrival Time, Burst Time, Completion Time, Waiting Time, and Turnaround Time.
22. Display the Average Waiting Time and Average Turnaround Time.

PROGRAM:

```
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];
    float total_wt = 0, total_tat = 0;

    printf("Enter Arrival Time and Burst Time for each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d - Arrival Time: ", i + 1);
        scanf("%d", &at[i]);
        printf("P%d - Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
    }
    for (i = 0; i < n; i++) {
        if (i == 0) {
            ct[i] = at[i] + bt[i];
        } else {
            if (at[i] > ct[i - 1]) {
                ct[i] = at[i] + bt[i];
            } else {
                ct[i] = ct[i - 1] + bt[i];
            }
        }

        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        total_wt += wt[i];
        total_tat += tat[i];
    }
    printf("\nProcess\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t\t%d\t\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage Turnaround Time: %.2f", total_tat / n);
    printf("\nAverage Waiting Time: %.2f\n", total_wt / n);

    return 0;
}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

OUTPUT:

TEST CASE1:

```
25071A6236\FCFS with
Enter number of processes: 5
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 0
P1 - Burst Time: 2
P2 - Arrival Time: 1
P2 - Burst Time: 3
P3 - Arrival Time: 2
P3 - Burst Time: 5
P4 - Arrival Time: 3
P4 - Burst Time: 4
P5 - Arrival Time: 4
P5 - Burst Time: 6

Process Arrival Burst   Completion      Turnaround      Waiting
P1      0       2       2               2               0
P2      1       3       5               4               1
P3      2       5       10              8               3
P4      3       4       14              11              7
P5      4       6       20              16              10

Average Turnaround Time: 8.20
Average Waiting Time: 4.20
```

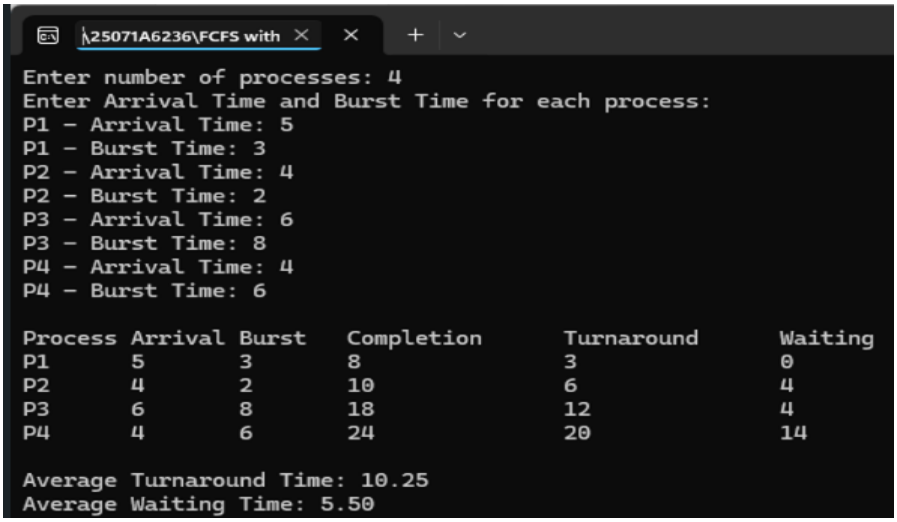
TESTCASE 2:

```
\25071A6236\FCFS with
Enter number of processes: 3
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 1
P1 - Burst Time: 2
P2 - Arrival Time: 2
P2 - Burst Time: 3
P3 - Arrival Time: 3
P3 - Burst Time: 4

Process Arrival Burst   Completion      Turnaround      Waiting
P1      1       2       3               2               0
P2      2       3       6               4               1
P3      3       4       10              7               3

Average Turnaround Time: 4.33
Average Waiting Time: 1.33
```

TEXT CASE 3:


\25071A6236\FCFS with
Enter number of processes: 4
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 5
P1 - Burst Time: 3
P2 - Arrival Time: 4
P2 - Burst Time: 2
P3 - Arrival Time: 6
P3 - Burst Time: 8
P4 - Arrival Time: 4
P4 - Burst Time: 6
Process Arrival Burst Completion Turnaround Waiting
P1 5 3 8 3 0
P2 4 2 10 6 4
P3 6 8 18 12 4
P4 4 6 24 20 14
Average Turnaround Time: 10.25
Average Waiting Time: 5.50

2) a) SJF (Short Job First) Scheduling algorithm non-preemptive.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT) and Burst Time (BT) for each process.
4. Assign process IDs (P1, P2, ..., Pn).
5. Initialize:
6. time = 0
7. completed = 0
8. Mark all processes as not completed.
9. While completed < n: Find all processes that have: Arrived (AT ≤ time), and Are not completed.
10. Among these processes, select the one with the smallest Burst Time (BT).
11. If two processes have the same Burst Time, choose the one with the earlier Arrival Time.
12. If no process has arrived, increment time by 1.
13. Execute the selected process until it completes (non-preemptive).
14. Calculate the Completion Time (CT): CT = time + BT
15. Update: time = CT
16. Mark the process as completed.
17. Calculate the Turnaround Time (TAT):
18. TAT = CT - AT
19. Calculate the Waiting Time (WT):WT = TAT - BT
20. Repeat Steps 6–11 until all processes are completed.
21. Average Waiting Time = ΣWT / n
22. Average Turnaround Time = ΣTAT / n
23. Display the Process ID, Arrival Time, Burst Time, Completion Time, Waiting Time, and Turnaround Time.
24. Stop

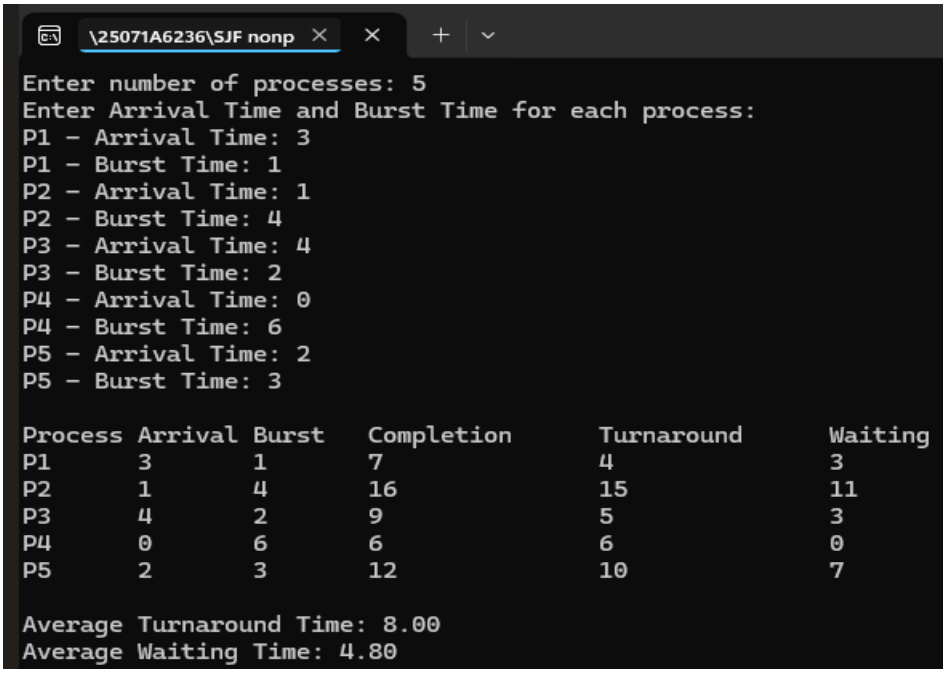
PROGRAM:

```
#include <stdio.h>
#include <stdbool.h>
int main() {
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int at[n], bt[n], ct[n], tat[n], wt[n];
    bool is_completed[n];
    float total_wt = 0, total_tat = 0;
   for (i = 0; i < n; i++) {
        is_completed[i] = false;
    }
    printf("Enter Arrival Time and Burst Time for each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d - Arrival Time: ", i + 1);
        scanf("%d", &at[i]);
        printf("P%d - Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
    }
    int current_time = 0;
    int completed = 0;
    while (completed < n) {
        int index = -1;
        int min_bt = 1e9;
        for (i = 0; i < n; i++) {
            if (at[i] <= current_time && !is_completed[i]) {
```

```
            if (bt[i] < min_bt) {
                min_bt = bt[i];
                index = i;
            }
            else if (bt[i] == min_bt) {
                if (at[i] < at[index]) {
                    index = i;
                } } } }
        if (index != -1) {
            ct[index] = current_time + bt[index];
            tat[index] = ct[index] - at[index];
            wt[index] = tat[index] - bt[index];
            total_wt += wt[index];
            total_tat += tat[index];
            is_completed[index] = true;
            completed++;
            current_time = ct[index];
        } else {
            current_time++;
        } }
    printf("\nProcess\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t\t%d\t\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    printf("\nAverage Turnaround Time: %.2f", total_tat / n);
    printf("\nAverage Waiting Time: %.2f\n", total_wt / n);
    return 0 ; }
```

OUTPUT:

TEST CASE 1:

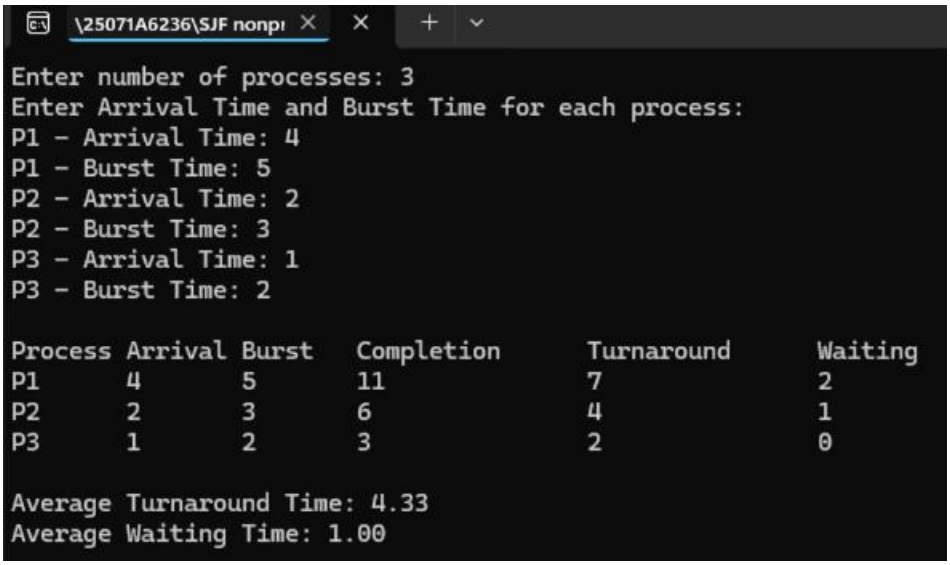


```
\25071A6236\SJF nonp
Enter number of processes: 5
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 3
P1 - Burst Time: 1
P2 - Arrival Time: 1
P2 - Burst Time: 4
P3 - Arrival Time: 4
P3 - Burst Time: 2
P4 - Arrival Time: 0
P4 - Burst Time: 6
P5 - Arrival Time: 2
P5 - Burst Time: 3

Process Arrival Burst   Completion      Turnaround      Waiting
P1      3       1       7               4               3
P2      1       4       16              15              11
P3      4       2       9               5               3
P4      0       6       6               6               0
P5      2       3       12              10              7

Average Turnaround Time: 8.00
Average Waiting Time: 4.80
```

TEST CASE 2:



```
\25071A6236\SJF nonp
Enter number of processes: 3
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 4
P1 - Burst Time: 5
P2 - Arrival Time: 2
P2 - Burst Time: 3
P3 - Arrival Time: 1
P3 - Burst Time: 2

Process Arrival Burst   Completion      Turnaround      Waiting
P1      4       5       11              7               2
P2      2       3       6               4               1
P3      1       2       3               2               0

Average Turnaround Time: 4.33
Average Waiting Time: 1.00
```

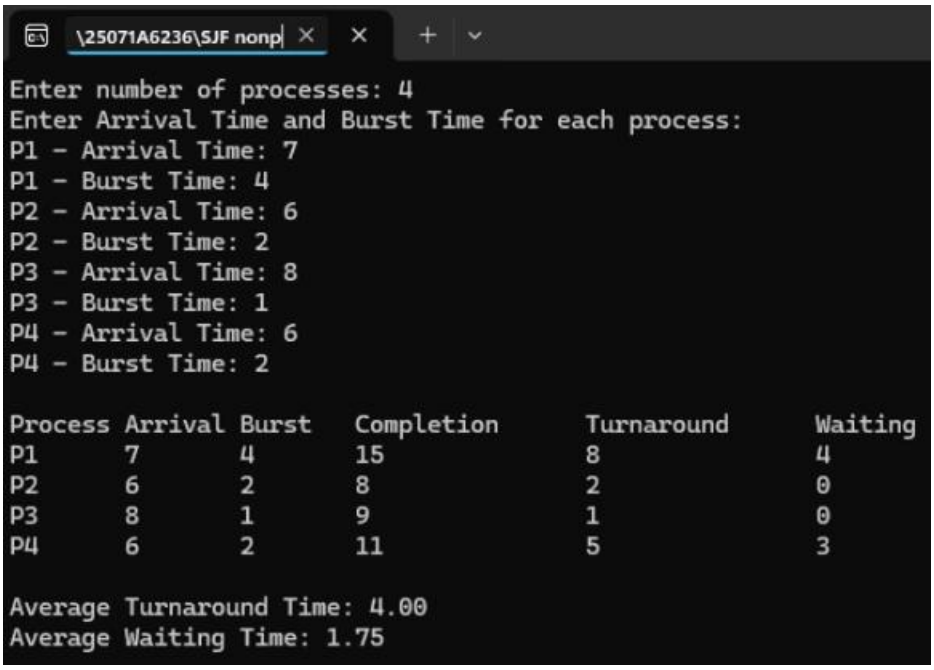
VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

TEST CASE 3:



```
\25071A6236\SJF nonp
Enter number of processes: 4
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 7
P1 - Burst Time: 4
P2 - Arrival Time: 6
P2 - Burst Time: 2
P3 - Arrival Time: 8
P3 - Burst Time: 1
P4 - Arrival Time: 6
P4 - Burst Time: 2

Process Arrival Burst   Completion      Turnaround      Waiting
P1      7       4       15              8               4
P2      6       2       8               2               0
P3      8       1       9               1               0
P4      6       2       11              5               3

Average Turnaround Time: 4.00
Average Waiting Time: 1.75
```

VNR VJIET

Name of the Laboratory: _____

Name of the Experiment: _____

Experiment No: _____ **Date:** _____

2) a) SJF (Short Job First) Scheduling algorithm preemptive OR (SRTF)

ALGORITHM:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT) and Burst Time (BT) for each process.
4. Assign process IDs (P1, P2, ..., Pn).
5. Initialize: Remaining Time (RT) = Burst Time (BT) for each process.
6. time = 0 AND completed = 0
7. Mark all processes as not completed.
8. While completed < n: Find all processes that have: Arrived (AT ≤ time), and Remaining Time (RT) > 0.
9. Among these, select the process with the smallest Remaining Time (RT).
10. If two processes have the same Remaining Time, choose the one with the earlier Arrival Time.
11. If no process has arrived, increment time by 1.
12. Execute the selected process for 1 unit of time.
13. Decrease its Remaining Time: RT = RT - 1
14. Increment the current time: time = time + 1
15. If the Remaining Time becomes 0:
16. Mark the process as completed. And Calculate Completion Time (CT): CT = time
17. Calculate Turnaround Time (TAT): TAT = CT - AT
18. Calculate Waiting Time (WT): WT = TAT - BT
19. Increment the completed process count.
20. Repeat Steps 6–10 until all processes are completed.
21. Calculate: Average Waiting Time = ΣWT / n
22. Average Turnaround Time = ΣTAT / n
23. Display the Process ID, Arrival Time, Burst Time, Completion Time, Waiting Time, and Turnaround Time.
24. Stop.

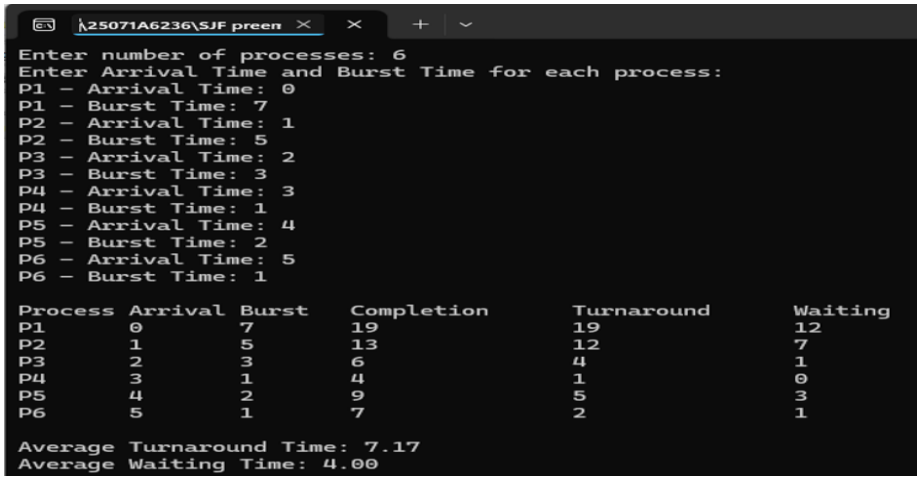
PROGRAM:

```
#include <stdio.h>
#include <stdbool.h>
int main() {
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];
    bool is_completed[n];
    float total_wt = 0, total_tat = 0;
    for (i = 0; i < n; i++) {
        is_completed[i] = false;
    }
    printf("Enter Arrival Time and Burst Time for each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d - Arrival Time: ", i + 1);
        scanf("%d", &at[i]);
        printf("P%d - Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    int current_time = 0;
    int completed = 0;
    int prev_index = -1;
    while (completed < n) {
        int index = -1;
        int min_rt = 1e9;
        for (i = 0; i < n; i++) {
```

```
            if (at[i] <= current_time && !is_completed[i]) {
                if (rt[i] < min_rt) {
                    min_rt = rt[i];
                    index = i;  }
                else if (rt[i] == min_rt) {
                    if (at[i] < at[index]) {
                        index = i;  }}}}
            if (index != -1) {
                rt[index]--;
                current_time++;
                if (rt[index] == 0) {
                    ct[index] = current_time;
                    tat[index] = ct[index] - at[index];
                    wt[index] = tat[index] - bt[index];
                    total_wt += wt[index];
                    total_tat += tat[index];
                    is_completed[index] = true;
                    completed++;} }
                else {
                 current_time++; } }
        printf("\nProcess\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\n");
        for (i = 0; i < n; i++) {
            printf("P%d\t%d\t%d\t%d\t\t%d\t\t%d\n", i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);  }
        printf("\nAverage Turnaround Time: %.2f", total_tat / n);
        printf("\nAverage Waiting Time: %.2f\n", total_wt / n);
        return 0;
    }
```

OUTPUT:

TESTCASE 1:

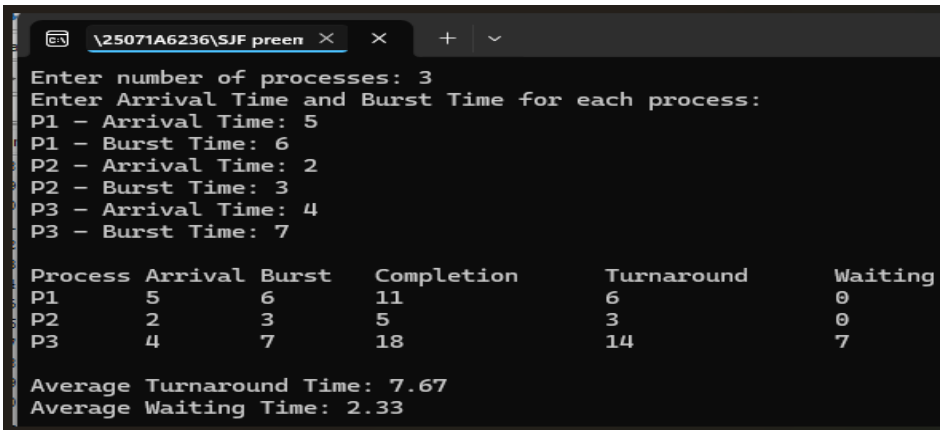


```
\25071A6236\SJF preem
Enter number of processes: 6
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 0
P1 - Burst Time: 7
P2 - Arrival Time: 1
P2 - Burst Time: 5
P3 - Arrival Time: 2
P3 - Burst Time: 3
P4 - Arrival Time: 3
P4 - Burst Time: 1
P5 - Arrival Time: 4
P5 - Burst Time: 2
P6 - Arrival Time: 5
P6 - Burst Time: 1

Process Arrival Burst   Completion      Turnaround      Waiting
P1      0       7       19              19              12
P2      1       5       13              12              7
P3      2       3       6               4               1
P4      3       1       4               1               0
P5      4       2       9               5               3
P6      5       1       7               2               1

Average Turnaround Time: 7.17
Average Waiting Time: 4.00
```

TESTCASE 2:

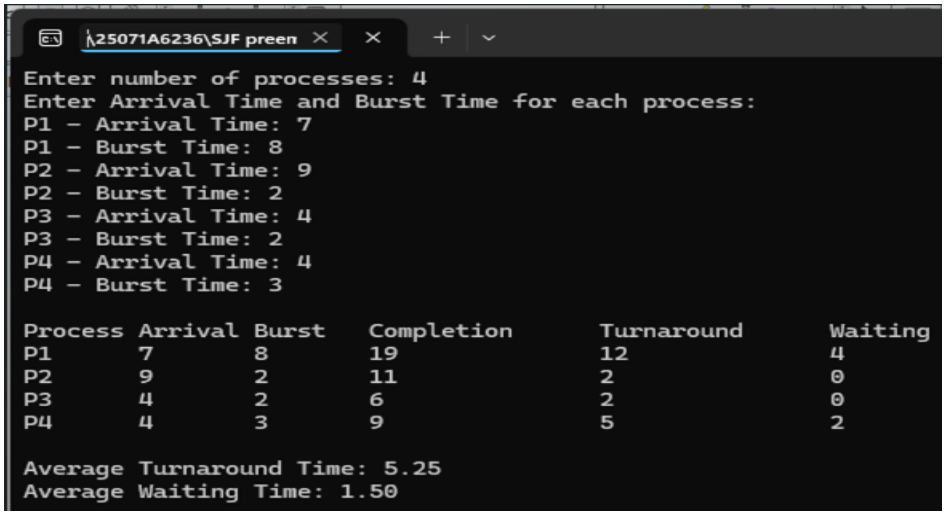


```
\25071A6236\SJF preem
Enter number of processes: 3
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 5
P1 - Burst Time: 6
P2 - Arrival Time: 2
P2 - Burst Time: 3
P3 - Arrival Time: 4
P3 - Burst Time: 7

Process Arrival Burst   Completion      Turnaround      Waiting
P1      5       6       11              6               0
P2      2       3       5               3               0
P3      4       7       18              14              7

Average Turnaround Time: 7.67
Average Waiting Time: 2.33
```

TEST CASE 3:



```
\25071A6236\SJF preem
Enter number of processes: 4
Enter Arrival Time and Burst Time for each process:
P1 - Arrival Time: 7
P1 - Burst Time: 8
P2 - Arrival Time: 9
P2 - Burst Time: 2
P3 - Arrival Time: 4
P3 - Burst Time: 2
P4 - Arrival Time: 4
P4 - Burst Time: 3

Process Arrival Burst   Completion      Turnaround      Waiting
P1      7       8       19              12              4
P2      9       2       11              2               0
P3      4       2       6               2               0
P4      4       3       9               5               2

Average Turnaround Time: 5.25
Average Waiting Time: 1.50
```